

Diffusion Models: Theory

From a known noising process back to data — predict the noise, then reverse it

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

**The generative problem
becomes denoising**

Why a new family of generative models?

We have met two ways to generate data:

VAE (L19) — organize a latent space, decode a latent in **one** pass. Trained by a likelihood bound.

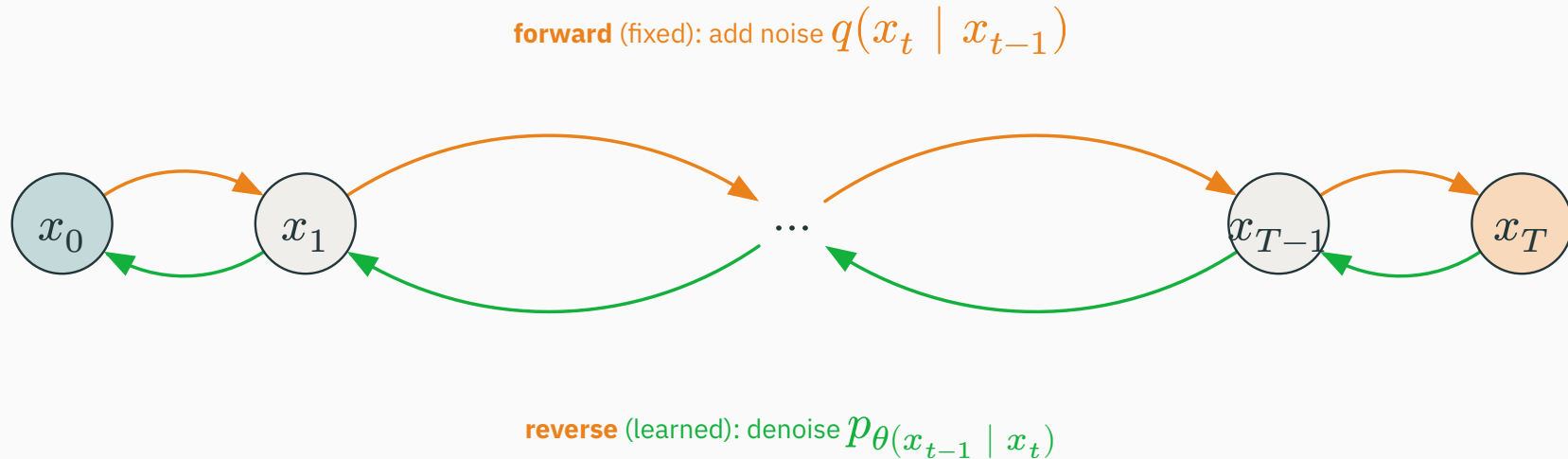
GAN (L20) — learn a generator against an **adversarial critic**. One pass, but unstable minimax training.

Diffusion starts from a **corruption process we design** — then generates by iterating a learned reverse process

the key move: if we **know** how to add noise, we can train a network to **remove** it

The one idea of this lecture

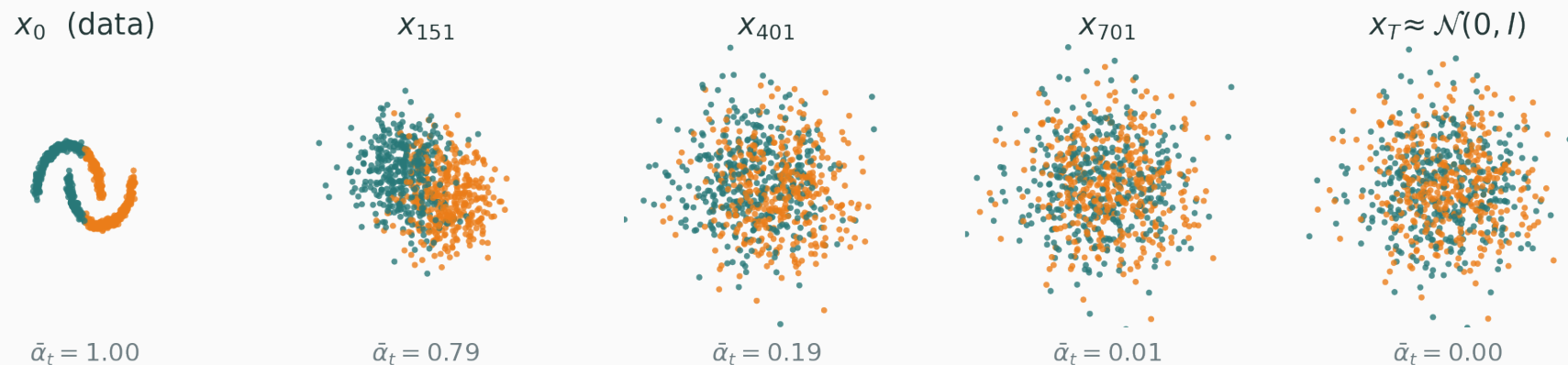
a diffusion model gradually turns **data** into **Gaussian noise**, then learns to run that sequence **backwards**, one small denoising step at a time.



forward is **fixed and known**; only the **reverse** transitions are **learned**

Visual hook: the noising filmstrip

Take a data point x_0 and add a little Gaussian noise, over and over:



forward process $q(x_t | x_{t-1})$ — **add a little Gaussian noise each step** →

structure dissolves smoothly; by $t = T$ the sample is **indistinguishable from pure noise**

Each forward step is a **fixed** Gaussian that shrinks the signal and injects noise:

$$q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

- $\beta_t \in (0, 1)$ is a small **variance schedule** — how much noise this step adds;
- nothing here is learned: the forward chain is **chosen in advance**.

$\sqrt{1 - \beta_t}$ **scales down** the previous sample; $\beta_t I$ **adds** fresh noise

Define, for each step and cumulatively:

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s$$

- α_t is the **fraction of variance kept** in one step;
- $\bar{\alpha}_t$ is the **fraction of the original signal kept** after t steps.

$\bar{\alpha}_t$ falls from ≈ 1 (early) toward ≈ 0 (late) — it is the whole story of the schedule

Read the forward step as two operations:

$$x_t = \underbrace{\sqrt{\alpha_t} x_{t-1}}_{\text{shrink the signal}} + \underbrace{\sqrt{\beta_t} \varepsilon}_{\text{add a little noise}}, \quad \varepsilon \sim \mathcal{N}(0, I)$$

each step barely changes the sample — many **tiny** steps accumulate into total corruption

The closed form that makes it all practical

Because Gaussians compose, we can jump straight to **any** noise level:

$$q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I)$$

no need to simulate steps $1, 2, \dots, t - 1$ — one formula reaches x_t directly

mean $\sqrt{\bar{\alpha}_t} x_0$ is the **surviving signal**; variance $(1 - \bar{\alpha}_t)I$ is the **accumulated noise**

The closed form is a one-line recipe with the **reparameterization trick**:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

- draw a clean x_0 , draw a noise ε , pick a timestep t ;
- the pair (x_t, ε) is a **ready-made supervised example** — input x_t , target ε .

we **know the noise** because we drew it ourselves — this is the crux of the whole method

Board calculation: build x_t by hand

Let $x_0 = 1$, $\bar{\alpha}_t = 0.64$, and a drawn noise $\varepsilon = -0.5$.

$$x_t = \sqrt{0.64}(1) + \sqrt{1 - 0.64}(-0.5)$$

$$x_t = \underbrace{0.8}_{\text{signal}} + \underbrace{0.6 \cdot (-0.5)}_{\text{noise}} = 0.8 - 0.3 = \mathbf{0.5}$$

signal $\sqrt{\bar{\alpha}_t} x_0 = 0.8$ — most of the clean value survives here

noise $\sqrt{1 - \bar{\alpha}_t} \varepsilon = -0.3$ — a moderate perturbation at this t

As $t \rightarrow T$, the product $\bar{\alpha}_t$ collapses toward zero:

$$\bar{\alpha}_T \approx 0 \implies q(x_T \mid x_0) \approx \mathcal{N}(0, I)$$

x_T is (almost) **standard Gaussian noise** — trivial to sample from

so generation can **start from pure noise** and only needs to learn the way **back**

Learning the reverse process

The direction that looks impossible

From one noisy image, **many** clean images are plausible:

given a noisy x_t , which x_{t-1} did it come from?

The reverse of a random corruption is **not a function** — it is a **distribution**. There is genuine uncertainty about what was destroyed.

so we model the reverse step **probabilistically**, and learn it from data

Model each reverse transition as a Gaussian whose mean (and variance) the network predicts:

$$p_{\theta}(x_{t-1} \mid x_t) = \mathcal{N}(\mu_{\theta(x_t, t)}, \Sigma_{\theta(x_t, t)})$$

- if steps are small, the **true** reverse step is **approximately Gaussian** too — so this form is well-motivated;
- all the difficulty is pushed into predicting the **mean** μ_{θ} .

the network must turn a slightly-noisier x_t into a slightly-cleaner x_{t-1}

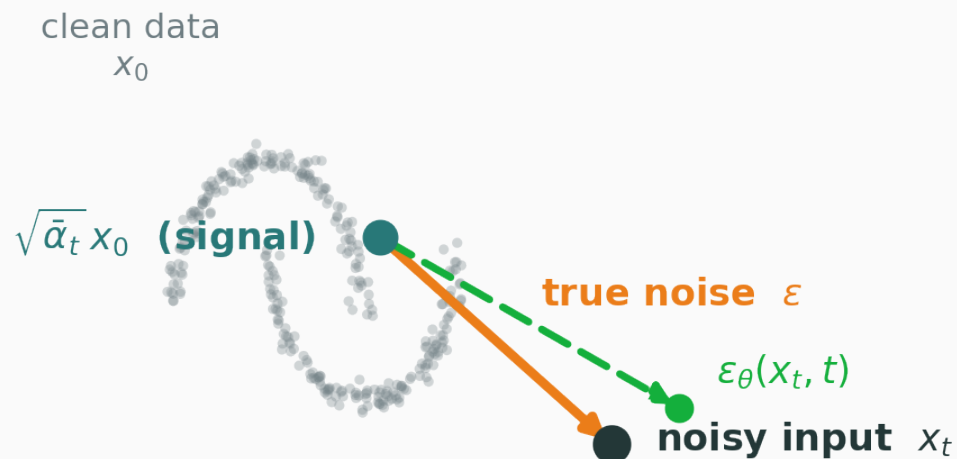
The task we can actually supervise

During training we **know both** x_0 and the noise ε we injected. So flip the problem:

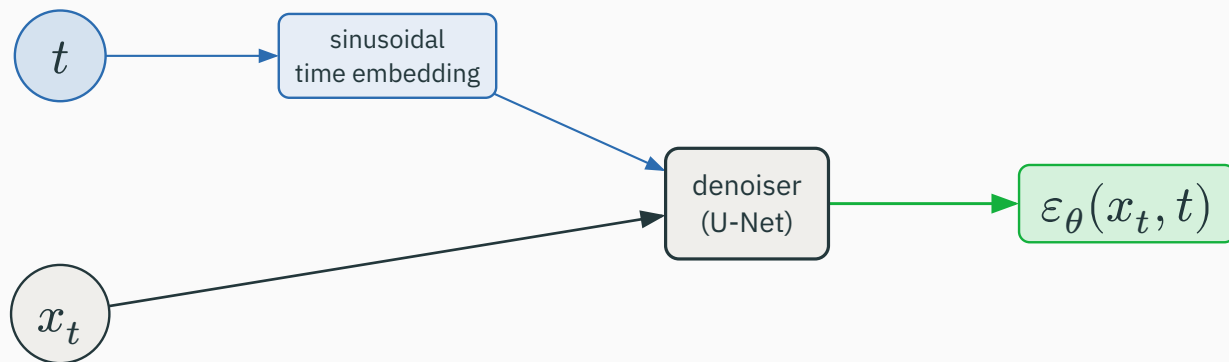
don't predict x_{t-1} directly — predict the noise ε that was added

The task we can actually supervise

the supervised target is the injected noise; train $\varepsilon_\theta(x_t, t) \approx \varepsilon$



A single network $\varepsilon_{\theta}(x_t, t)$ estimates the injected noise from the noisy input and the timestep:



it **must** see t : “remove a little noise” and “remove almost all noise” are different jobs

Train by simple regression of predicted noise onto true noise:

$$L_{\text{simple}} = \mathbb{E}_{x_0, \varepsilon, t} \left[\left\| \varepsilon - \varepsilon_{\theta(x_t, t)} \right\|_2^2 \right]$$

- expectation over data x_0 , injected noise ε , and timestep t ;
- this is **supervised regression** with **self-generated** targets — no labels, no adversary.

the whole of DDPM training is: **sample noise, predict it, minimize the MSE**

A useful reframe: self-supervised denoising

Every clean example is an **infinite** source of training pairs:

one clean x_0 $\rightarrow$ pick any t , draw any ε
 $\rightarrow$ a fresh noisy input x_t .

exact target — the noise ε is **known by construction**, so the label is free and precise.

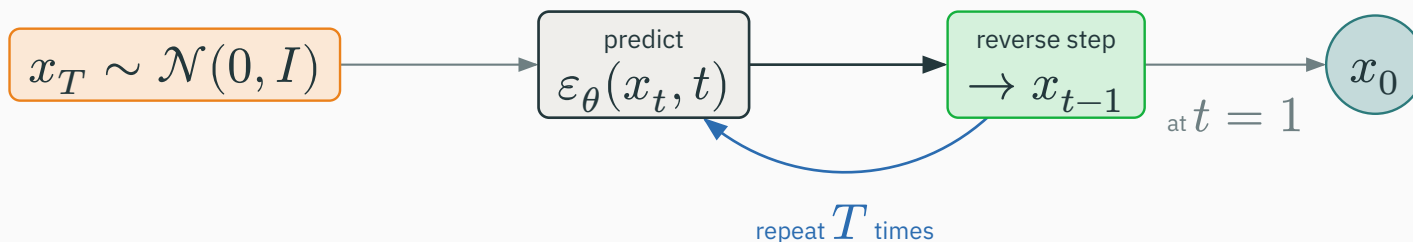
diffusion is denoising self-supervision at **every** noise level at once

Invert the closed form: if we can guess ε , we can guess the clean signal.

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_{\theta(x_t, t)}}{\sqrt{\bar{\alpha}_t}}$$

predicting the noise and predicting the clean image are **two views of the same quantity**

Start at pure noise; peel off a little noise at a time until a sample emerges:



```
x_T ~ N(0, I)                # start from pure Gaussian noise
for t = T, ..., 1:
    eps_hat = eps_theta(x_t, t)  # predict the noise
    x_{t-1} = reverse_step(x_t, eps_hat, t)  # sample / estimate x_{t-1}
return x_0
```

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/diffusion-denoise

Slide the noise level to watch $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$ corrupt an example, then run the reverse chain step-by-step and see a sample condense out of noise.

Why generation is slow

The cost of quality is **many** forward passes:

GAN — one z , one generator pass, one image. Fast.

Diffusion — the **same** denoiser is called **hundreds to thousands** of times per sample.

stable training and broad coverage — paid for in **inference** time (L22 attacks this)

Interpreting the geometry

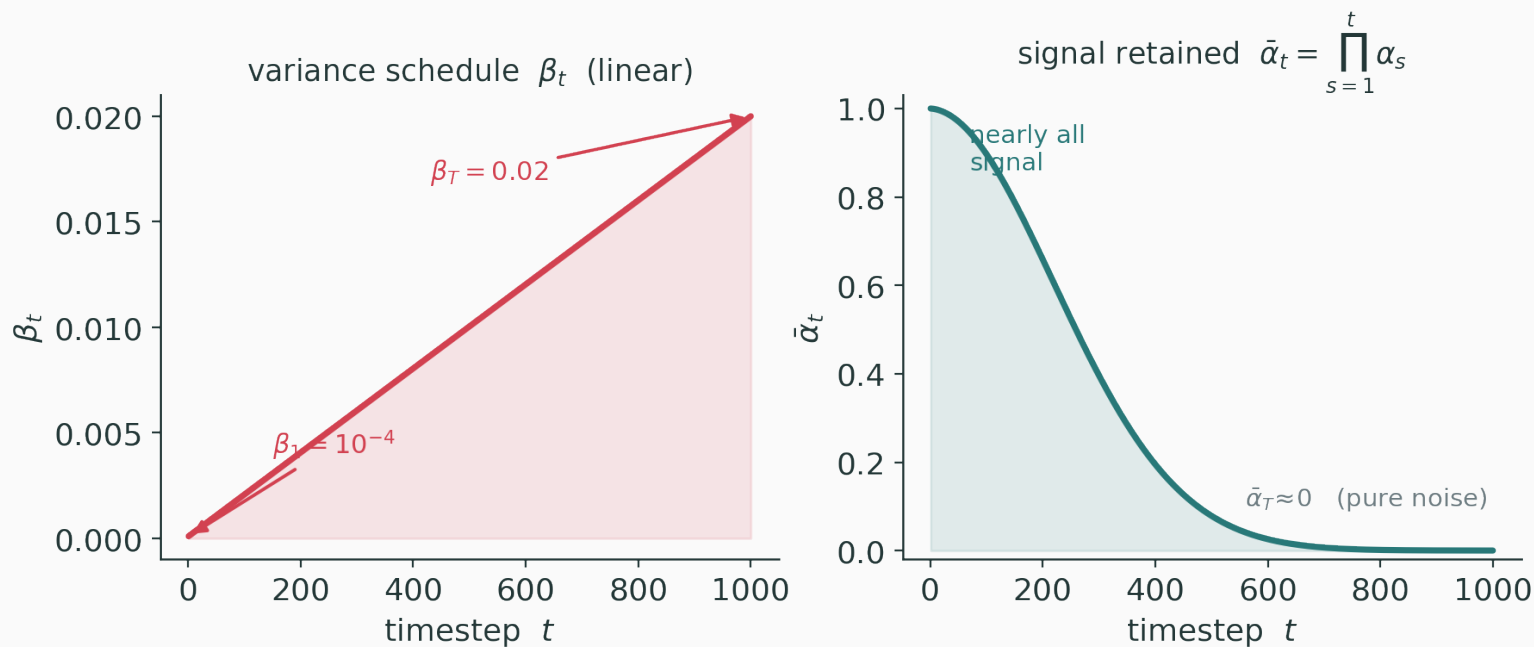
Different reverse steps do qualitatively different work:

- **early** reverse steps (near pure noise) — recover **global layout** and coarse structure;
- **late** reverse steps (nearly clean) — restore **fine texture** and detail.

one network, but the **job changes** along the trajectory — hence the time conditioning

The noise schedule is a curriculum

Training touches **every** difficulty, from trivial to nearly hopeless:



β_t (how much noise each step adds) sets $\bar{\alpha}_t$ (how much signal survives)

For a noisy distribution p_t , the **score** is the gradient of its log-density:

$$\nabla_x \log p_t(x) \quad (\text{points toward } \textbf{higher-density} \text{ regions})$$

Predicting Gaussian noise is **equivalent** (up to a schedule factor) to estimating this score:

$$\varepsilon_{\theta(x_t, t)} \approx -\sqrt{1 - \bar{\alpha}_t} \nabla_x \log p_t(x_t).$$

a bridge to **score-based** models — we stop here, no SDE derivation

What the network actually learns

Not a lookup table of clean images:

it learns, from any noisy point, a **local direction** toward where the data density is high

a **vector field of denoising directions** — generalized across inputs, timesteps, and conditions

Feed the denoiser extra information c so the trajectory follows a desired target:

- a **class label**; a **text embedding**; a **segmentation mask**; a **low-resolution image**;

$\varepsilon_{\theta}(x_t, t, c)$ (same network, now told what to aim for)

the denoising **question** is unchanged — only the **conditioning** differs (full story in L22)

Three generative families so far

Family	Training signal	Sampling
VAE	likelihood (ELBO) bound	one latent decode
GAN	adversarial critic	one generator pass
Diffusion	predict known injected noise	repeated denoising

diffusion trades a **hard sampler** for an **easy, stable** learning signal

What diffusion gains and what it pays

gains — stable supervised-style training; broad mode coverage; flexible conditioning.

pays — many inference steps; substantial compute per sample.

the central trade of the method: **training ease** against **sampling cost**

If t is **large** (a very noisy input), should the model rely more on the **noisy input** or on its **learned data prior**? Why?

At large t , x_t carries **almost no signal** ($\bar{\alpha}_t \approx 0$). The denoiser must lean on the **prior** it learned — the input mostly tells it **which** broad direction, not the fine details.

Theory says: **predict the noise, repeatedly.**

Practice then asks three questions:

1. **what architecture** predicts ε_θ ? (U-Net / diffusion Transformer)
2. **how does text steer it?** (conditioning + guidance)
3. **can we avoid thousands of steps?** (fast samplers, latent space)

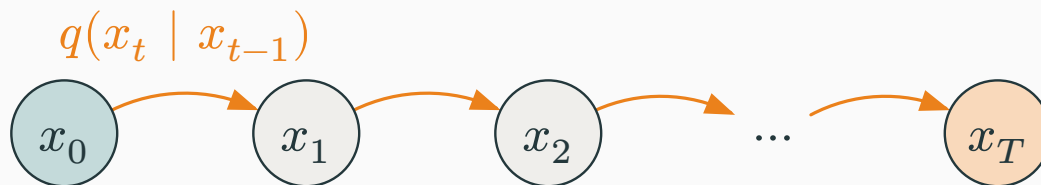
L22 answers all three

Diffusion theory in depth

The forward chain factorizes

The forward process is a **Markov chain** — each step depends only on the last:

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^T q(x_t \mid x_{t-1})$$

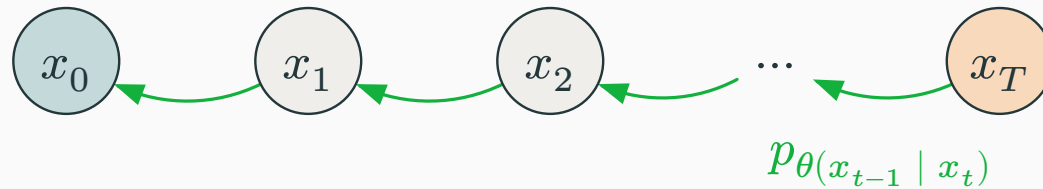


a simple, fixed structure — even when the data distribution is wildly complicated

The reverse model factorizes too

Sampling runs the learned chain from the easy prior downward:

$$p_{\theta}(x_{0:T}) = p(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1} | x_t), \quad p(x_T) = \mathcal{N}(0, I)$$



start at $\mathcal{N}(0, I)$, apply T learned reverse transitions, arrive at x_0

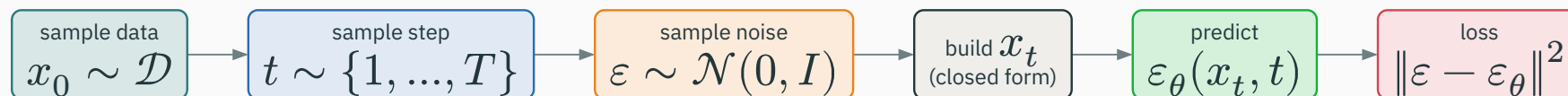
Why the closed form matters computationally

Without it, one training pair at timestep t would need t **sequential** noising steps:

naive — to get x_t , simulate $x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_t$. Costly, and different for every t .

closed form — $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon$
in **one** operation, for **any** t .

one random t gives one training example in a **single** step — this is what makes training feasible

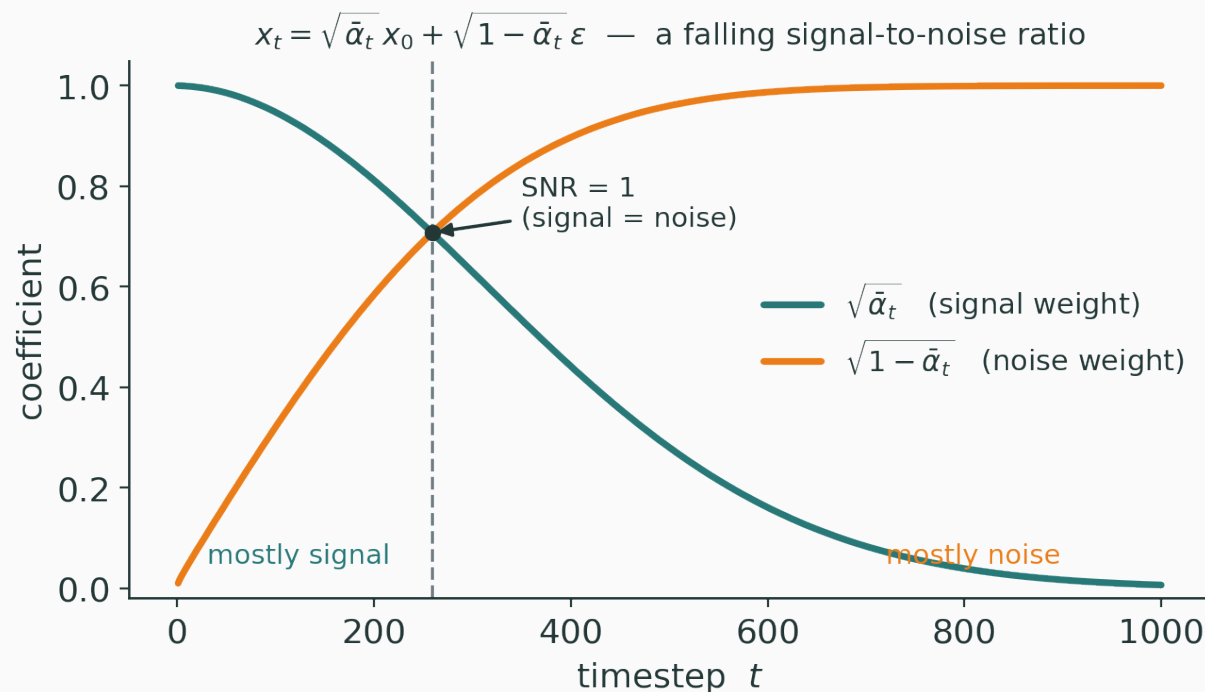


The five operations of one training step:

sample $x_0 \rightarrow$ sample $t \rightarrow$ sample $\varepsilon \rightarrow$ build $x_t \rightarrow$ predict $\varepsilon_{\theta(x_t, t)}$

the same data point spawns a **continuum** of self-supervised denoising tasks

Plot the signal and noise coefficients against t :



$\sqrt{\bar{\alpha}_t}$ falls from 1; $\sqrt{1 - \bar{\alpha}_t}$ rises to 1 — a steadily **falling signal-to-noise ratio**

Different timesteps are different tasks

Timestep	Input appearance	Main denoising demand
small t	mostly clean	remove fine perturbations
middle t	partly structured	recover objects / layout
large t	near Gaussian noise	rely strongly on the learned prior

the time embedding tells the network **which** of these regimes it is in

Why predict noise rather than the image?

Several equivalent parameterizations exist:

ε (noise), x_0 (clean image), v (velocity)

- **noise prediction** gives a target with a **known, unit-scale** Gaussian across all t — nicely conditioned;
- practical systems may use x_0 or v ; the underlying diffusion principle is **unchanged**.

the parameterization is a **modeling choice**, not the essence of diffusion

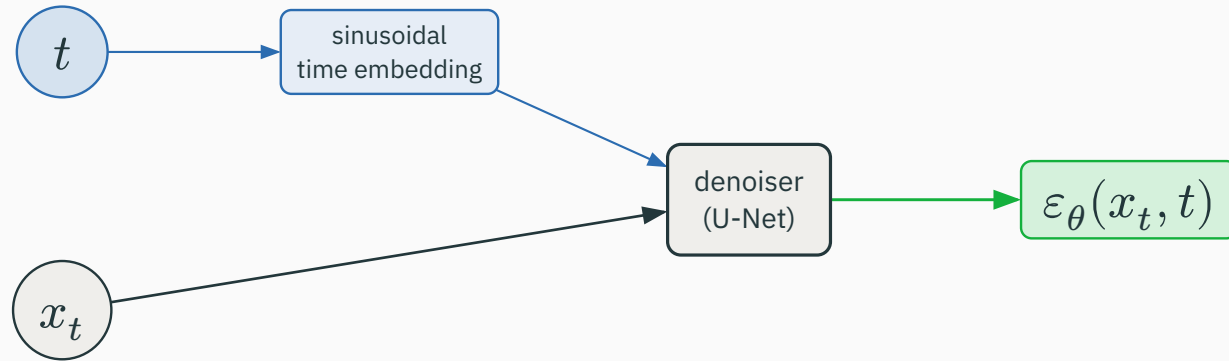
From an ambiguous x_t , the network cannot recover the **exact** unknowable noise draw:

$$\varepsilon_{\theta(x_t, t)} \rightarrow \mathbb{E}[\varepsilon \mid x_t, t]$$

Minimizing MSE makes ε_{θ} approach the **conditional mean** of the noise given x_t . It blends the evidence in x_t with the **data prior** — exactly what a good denoiser should do.

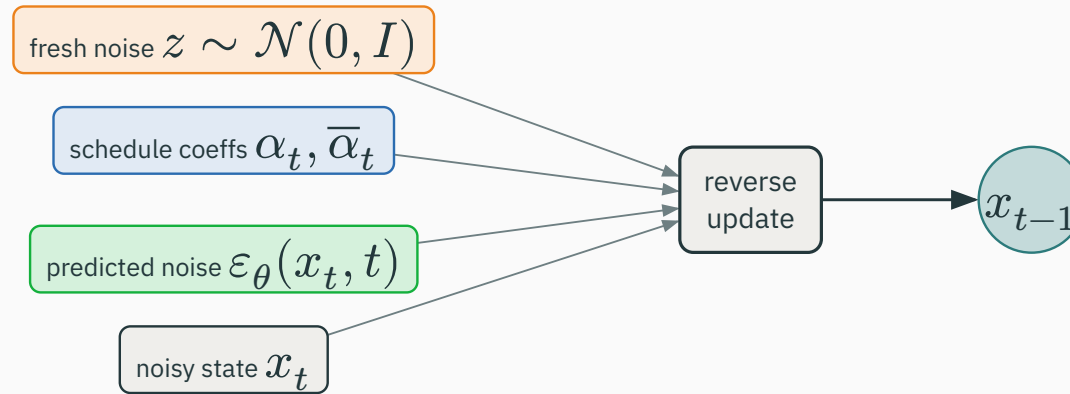
Time must enter the network

The **same** tensor can occur at many noise levels — the model needs to know which:



$t \rightarrow$ sinusoidal / learned embedding $\rightarrow$ injected into the denoiser blocks

One reverse step **combines** four ingredients — no need to memorize the coefficients:



current state, predicted noise, schedule, and (optionally) fresh randomness $\rightarrow x_{t-1}$

Once the denoiser is trained, **how** you step back is a separate choice:

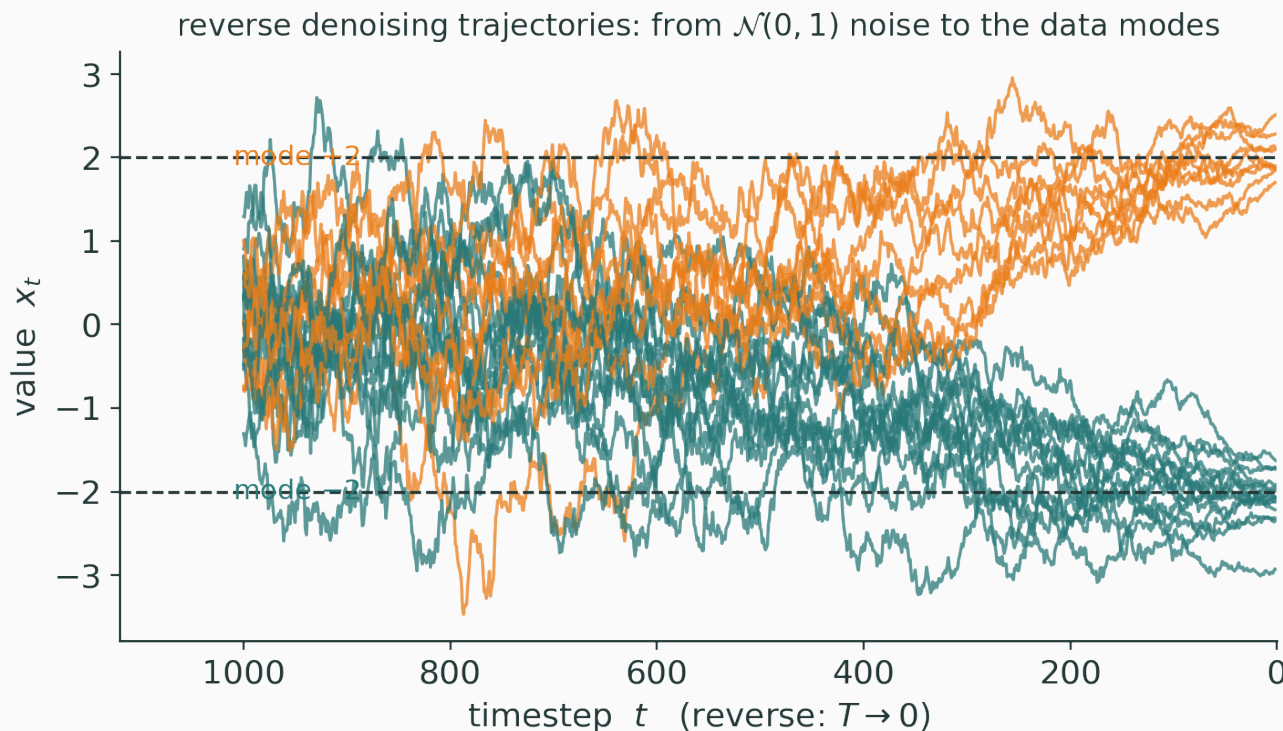
stochastic (e.g. DDPM) — add fresh noise z at each reverse step; more **diverse** samples.

deterministic (e.g. DDIM) — no added noise once x_T is fixed; **reproducible**, often **fewer** steps.

this changes diversity, reproducibility, and speed — **not** the learned denoiser itself

One sample, many intermediate states

Run the reverse chain and watch a value settle onto a data mode:



global choice (“which mode?”) is made **early**; fine position is refined **late** — an observation, not a law

A denoising autoencoder learns one corruption level:

corrupted input $\rightarrow$ clean input

diffusion is a DAE at every calibrated noise level, plus a generative reverse chain

the extra ingredients: a **sequence** of levels, **time conditioning**, and **sampling from noise**

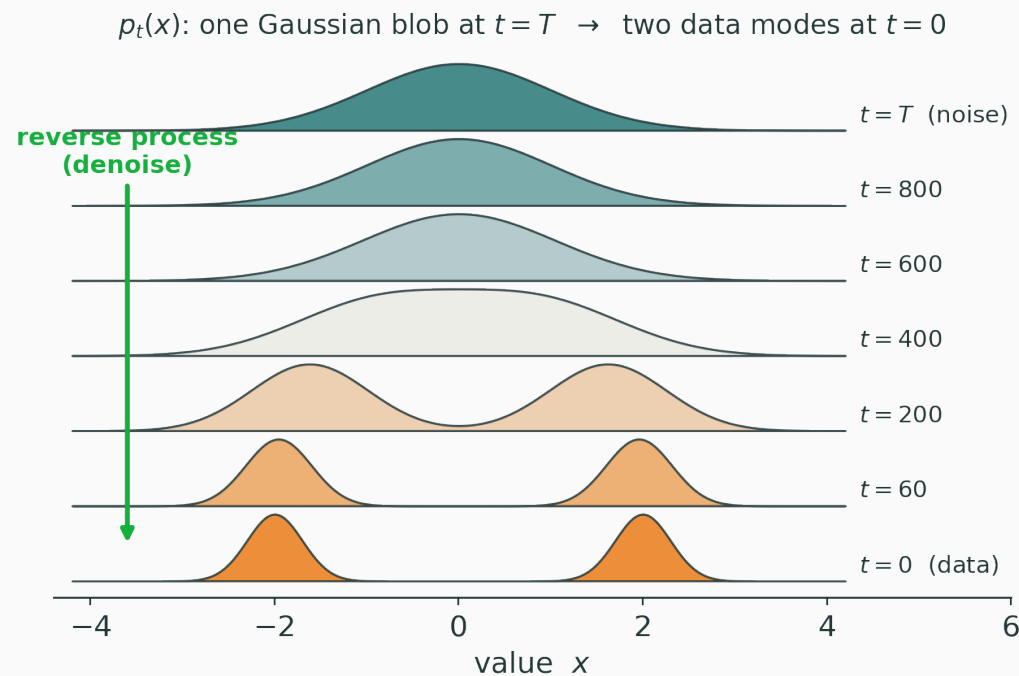
The original DDPM objective comes from a **variational bound** on the data likelihood:

$$-\log p_{\theta}(x_0) \leq \mathbb{E}_q[\dots] \quad (\text{a sum of per-step KL terms — like L19's ELBO})$$

The simple noise-prediction MSE L_{simple} is a **reweighted simplification** of that likelihood bound — it drops constants and per-timestep weights that hurt sample quality.

connect to the ELBO from L19 — we do **not** re-derive the whole bound here

For a noisy point x_t , the score points toward denser regions of p_t :



the reverse process rides these directions from one blurry blob back to the data modes

A condition c reshapes **which** reverse paths are favored:

$$p_{\theta}(x_{t-1} \mid x_t, c)$$

text, class labels, masks, low-res images — all change the **same** conditional denoising question

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/classifier-guidance

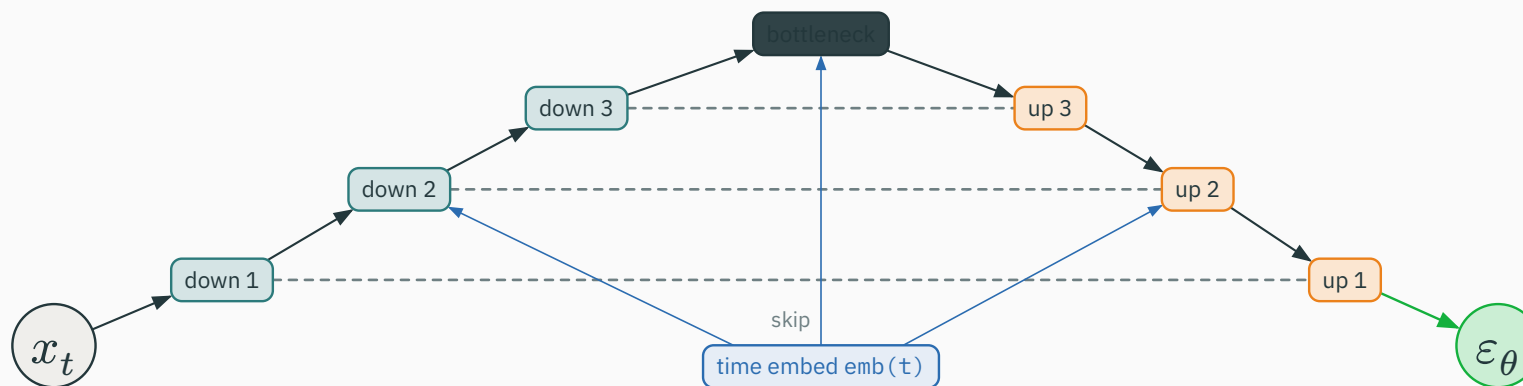
A conditioning widget: watch the reverse trajectory bend toward a chosen class.

An external classifier can push the trajectory toward a target class:

$$\nabla_{x_t} \log p(c \mid x_t) \quad (\text{gradient that says “look more like class } c\text{”})$$

add this to the score to **steer** sampling — L22 teaches the practical **classifier-free** version

The loss says **what** to predict; the architecture says **how** to compute it:



a **U-Net** (or diffusion Transformer): multi-scale, skip connections, time embedding in every block

Why diffusion training is comparatively stable

Every example has a **fixed, known** target:

$\text{target} = \varepsilon$ (the noise we drew — it never moves)

Unlike a GAN, **no learned opponent** shifts the target between batches — there is no minimax game to diverge.

optimization is not **trivial**, but the central instability of adversarial training is **gone**

Why diffusion sampling is expensive

Model family	Typical generation structure
VAE	one decode
GAN	one generator pass
diffusion	repeated denoiser calls (T of them)

the model trades **inference work** for a **tractable, stable** training signal

Misconception	Correction
diffusion memorizes a reverse “noise table”	a network generalizes denoising across inputs and conditions
the forward process is learned	in standard DDPMs it is chosen and fixed
the network only predicts the clean image	noise, clean image, or velocity can be parameterized
more steps always mean a better image	schedules and samplers set a quality / latency trade-off

1. Why can x_t be sampled **directly** from x_0 ?
2. What target is available during diffusion training **without labels**?
3. Why **must** the denoiser know t ?
4. Why is generation **slower** than GAN sampling?

if all four are quick to answer, the core of the lecture is secure

The β_t schedule is a design choice

The schedule sets the **sequence of learning problems**:

too little noise — x_T stays structured;
the prior $\mathcal{N}(0, I)$ is a poor starting
point.

too abrupt noise — intermediate
denoising tasks become needlessly
hard.

smooth schedules (linear, cosine) spread difficulty **evenly** across timesteps

The learned reverse Gaussian has a mean **and** a variance:

$$p_{\theta}(x_{t-1} | x_t) = \mathcal{N}(\mu_{\theta}, \Sigma_{\theta})$$

- Σ_{θ} may be **fixed** to a schedule value or **learned**;
- it controls how much randomness each reverse step injects.

variance is a **modeling knob**, not an arbitrary visual-effect dial

An “MSE loss” still hides modeling choices:

- not every t need contribute **equally** to the objective;
- parameterization ($\varepsilon / x_0 / v$) and schedule change **which** SNR regimes dominate learning.

reweighting timesteps trades off **coarse structure** against **fine detail** quality

The same recipe travels — only the denoiser and conditioning change:

- **audio** waveforms or spectrograms; **video** with temporal structure;
- **molecular** coordinates; **trajectories** and actions; continuous **latent** representations.

for standard continuous-data DDPMs, forward = Gaussian corruption; the **architecture** must respect the data's structure

Why direct pixel diffusion is demanding

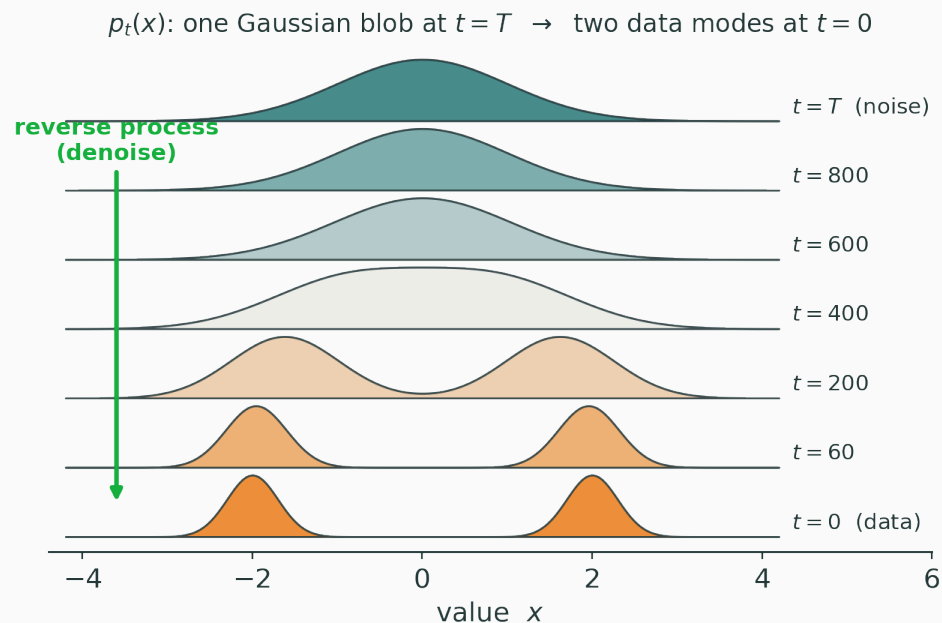
Noise is added independently to a **very high-dimensional** tensor:

a good denoiser must fix texture, object identity, global layout, and condition alignment at once

this pressure motivates L22's **latent-space** diffusion and multi-scale architectures

A tiny one-dimensional thought experiment

Data lives near -2 and $+2$ on a line; at high noise the two modes **blend**:



the reverse model must **decide which mode** a noisy point should approach — generation is inherently probabilistic

Denoising is not a nearest-neighbour lookup

For a noisy point between modes, the model uses **learned structure**, not a database:

- the result may resemble **interpolation**, **recombination**, or a genuinely **new** sample;

Separately: **memorization** remains an empirical risk to **test for** on small or narrow datasets.

No single grid or scalar settles quality — evaluate along five axes:

- **fidelity; coverage / diversity; conditioning adherence;**
- **sampling cost; robustness and harmful failure modes.**

FID and a caption match tell you **something** — never **everything**

Before reading any implementation, locate it with five questions:

1. data space — **pixels or latents**?
2. **denoiser architecture**?
3. **time / noise parameterization**?
4. **condition interface**?
5. **sampler** and step budget?

these five coordinates place nearly **every** diffusion system in the course map

Idea	Equation
forward step	$q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$
closed form	$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$
training loss	$L = \mathbb{E} \left\ \varepsilon - \varepsilon_{\theta(x_t, t)} \right\ _2^2$
clean estimate	$\hat{x}_0 = (x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_{\theta}) / \sqrt{\bar{\alpha}_t}$

a final reference — **not** another derivation

1. Which part of diffusion is **fixed** and which part is **learned**?
2. How does the closed form turn one image into **many** training examples?
3. Why is the reverse direction **conditional and probabilistic**?
4. What trade-off does **iterative sampling** introduce?

noise-prediction objective

+ latent space, U-Net / Transformer, and conditioning

= a **practical** diffusion system

L22 builds that system component by component — including **Stable Diffusion**

If you know how to **add** noise, you can train a network to **remove** it.

Next: **Diffusion in practice** → latent diffusion, U-Nets, guidance, and Stable Diffusion.